

CRM INTELIGENTTE - Planejamento de Produto e Execucao

Data: 01/07/2026

Visao Geral

O CRM INTELIGENTTE sera uma plataforma central para relacionamento com clientes, gestao comercial, demandas de clientes, demandas internas, automacoes e integracoes com ferramentas externas como n8n e Make.

O objetivo nao e criar apenas um cadastro de clientes. O objetivo e construir um sistema operacional da empresa: uma fonte unica de verdade sobre clientes, oportunidades, tarefas, demandas, historico, responsaveis, SLAs, automacoes e indicadores.

Decisoese Tecnicas Iniciais

- Produto: CRM INTELIGENTTE.
- Banco de dados: Neon com PostgreSQL.
- Aplicacao: Next.js, React, TypeScript.
- UI: Tailwind CSS e shadcn/ui.
- ORM recomendado: Drizzle ORM, por dar controle alto sobre SQL, migrations e tipagem.
- Autenticacao recomendada: Better Auth, com suporte a organizacoes, sessoes e permissoes customizadas.
- Deploy recomendado: Vercel para aplicacao web.
- Integracoes: API REST interna, webhooks outbound e endpoints inbound para n8n e Make.
- Jobs e automacoes internas: iniciar simples com eventos persistidos no banco; evoluir para fila/worker quando houver volume.

Principios do Produto

- Personalizacao primeiro: campos customizados, pipelines customizados, status customizados, tags e visualizacoes salvas.
- Separacao clara entre demandas de clientes e demandas internas.
- Integracao como requisito nativo: tudo que importa deve emitir evento e poder ser atualizado via API.
- Rastreabilidade: toda mudanca relevante deve gerar historico/auditoria.
- Operacao diaria eficiente: telas densas, filtraveis, com comandos rapidos e pouca friccao.
- Multiempresa desde cedo: base preparada para organizacoes, equipes, papeis e permissoes.

Modulos do Sistema

1. Fundacao

- Organizacoes.
- Usuarios.
- Equipes.
- Papeis e permissoes.
- Sessao e autenticao.
- Configuracoes da empresa.
- Auditoria basica.

2. Gestao de Clientes

- Clientes/empresas.
- Contatos.
- Historico de relacionamento.
- Notas.
- Arquivos.
- Status do cliente.
- Responsavel pelo cliente.
- Tags e campos personalizados.

3. Comercial

- Leads.
- Oportunidades.
- Pipelines.
- Etapas do funil.
- Atividades comerciais.
- Conversao de lead em cliente.

4. Demandas de Clientes

- Demandas vinculadas a cliente.
- Status, prioridade, responsavel e prazo.
- Comentarios internos.
- Checklists.
- Anexos.
- SLA.
- Historico de mudancas.

5. Demandas Internas

- Demandas sem vinculo obrigatorio com cliente.
- Areas/equipes internas.
- Projetos internos.
- Checklists, anexos, comentarios e prazos.
- Quadros por equipe.

6. Distribuicao de Demandas

- Atribuicao manual.
- Atribuicao por equipe.
- Distribuicao por rodada.
- Distribuicao por carga de trabalho.
- Regras simples configuraveis.

7. Automacoes e Integracoes

- Webhooks de saida para n8n e Make.
- API para n8n e Make criarem ou atualizarem dados.
- Eventos internos.
- Logs de execucao.
- Chaves de API.
- Teste de webhook.

8. Dashboard e Relatorios

- Visao geral de clientes.
- Demanda por status.
- Demandas vencidas.
- Responsaveis mais carregados.
- Funil comercial.
- Atividades recentes.

Roadmap em Sprints

Cada sprint considera um ciclo de 1 a 2 semanas, com entregas pequenas, testaveis e utilizaveis.

Sprint 0 - Preparacao Tecnica e Produto

Objetivo: preparar a base do projeto, decisoes tecnicas, ambientes e contratos iniciais.

US-0001 - Como fundador, quero uma base tecnica padronizada para iniciar o CRM com seguranca

Tasks:

- Criar repositorio Git.
- Criar projeto Next.js com TypeScript.
- Configurar Tailwind CSS e shadcn/ui.
- Configurar ESLint, Prettier e scripts de qualidade.
- Configurar variaveis de ambiente.
- Criar projeto Neon e obter DATABASE_URL.
- Configurar Drizzle ORM e primeira migration.
- Criar documentacao inicial de setup.

Critérios de aceitacao:

- O projeto roda localmente.
- Existe uma conexao funcional com Neon em ambiente de desenvolvimento.
- Existe script de migration.
- Existe documentacao de como iniciar o projeto.
- Variaveis sensiveis nao ficam versionadas.

BDD:

Cenario: Iniciar o projeto localmente
Dado que o desenvolvedor possui as variaveis de ambiente configuradas
Quando executar o comando de desenvolvimento
Entao a aplicacao deve abrir localmente sem erros criticos
E deve conseguir se conectar ao banco Neon

Sprint 1 - Autenticacao, Organizacoes e Usuarios

Objetivo: permitir login seguro e separar os dados por organizacao.

US-0101 - Como usuario, quero acessar o CRM com login seguro

Tasks:

- Implementar tela de login.
- Implementar logout.
- Configurar Better Auth.
- Criar tabela de usuarios.
- Criar protecao de rotas autenticadas.
- Criar layout base da aplicacao.

Critérios de aceitacao:

- Usuario autenticado acessa o painel.
- Usuario nao autenticado e redirecionado para login.
- Sessao expirada exige novo login.
- Logout encerra a sessao.

BDD:

Cenario: Acessar area protegida sem login
Dado que estou sem sessao ativa
Quando tento acessar o painel do CRM
Entao devo ser redirecionado para a tela de login

US-0102 - Como administrador, quero gerenciar usuarios da minha empresa

Tasks:

- Criar modelo de organizacao.
- Criar vinculo usuario-organizacao.
- Criar papeis iniciais: admin, gestor, membro.
- Criar tela simples de usuarios.

- Permitir convidar usuário por email em versão inicial.

Critérios de aceitação:

- Admin visualiza usuários da própria organização.
- Admin pode convidar novo usuário.
- Usuários de uma organização não visualizam dados de outra.
- Papéis iniciais são salvos no banco.

BDD:

```
Cenário: Admin visualiza usuários da organização
  Dado que sou admin da organização A
  Quando acesso a tela de usuários
  Então devo ver somente usuários da organização A
```

Sprint 2 - Gestão de Clientes e Contatos

Objetivo: criar o núcleo de relacionamento com clientes.

US-0201 - Como usuário, quero cadastrar clientes para centralizar informações

Tasks:

- Criar tabela de clientes.
- Criar campos: nome, documento, email, telefone, site, status, responsável, observações.
- Criar listagem com busca e filtro por status.
- Criar tela de criação e edição.
- Criar detalhe do cliente.
- Registrar histórico de criação e alteração.

Critérios de aceitação:

- Usuário pode criar cliente.
- Usuário pode editar cliente.
- Usuário pode listar e buscar clientes.
- Cliente criado fica vinculado à organização correta.
- Alterações relevantes geram histórico.

BDD:

```
Cenário: Criar novo cliente
  Dado que estou autenticado em uma organização
  Quando cadastro um cliente com nome e dados válidos
  Então o cliente deve aparecer na listagem
  E deve estar vinculado à minha organização
```

US-0202 - Como usuário, quero cadastrar contatos de um cliente

Tasks:

- Criar tabela de contatos.
- Vincular contatos a clientes.

- Criar campos: nome, cargo, email, telefone, principal.
- Criar area de contatos no detalhe do cliente.
- Permitir marcar contato principal.

Critérios de aceitacao:

- Cliente pode ter varios contatos.
- Apenas um contato principal deve existir por cliente.
- Contatos aparecem no detalhe do cliente.

BDD:

```
Cenario: Definir contato principal
  Dado que um cliente possui dois contatos
  Quando marco um contato como principal
  Entao o contato anterior deixa de ser principal
  E o novo contato passa a ser o principal
```

US-0203 - Como usuario, quero adicionar notas ao cliente

Tasks:

- Criar tabela de notas.
- Vincular notas a cliente e autor.
- Exibir linha do tempo no detalhe do cliente.
- Permitir criar e editar nota propria.

Critérios de aceitacao:

- Nota criada aparece na linha do tempo.
- Nota registra autor e data.
- Nota pertence a organizacao correta.

BDD:

```
Cenario: Registrar nota no cliente
  Dado que estou no detalhe de um cliente
  Quando adiciono uma nota
  Entao a nota deve aparecer no historico do cliente
```

Sprint 3 - Demandas de Clientes

Objetivo: organizar solicitacoes, tickets e tarefas vinculadas a clientes.

US-0301 - Como usuario, quero criar demanda vinculada a um cliente

Tasks:

- Criar tabela de demandas.
- Adicionar tipo client_demand.
- Campos: titulo, descricao, status, prioridade, prazo, responsavel, cliente.
- Criar listagem de demandas de clientes.
- Criar detalhe da demanda.

- Vincular demanda ao detalhe do cliente.

Critérios de aceitação:

- Demanda de cliente exige cliente vinculado.
- Demanda aparece na listagem geral e no detalhe do cliente.
- Status inicial padrão e configurado.
- Responsável pode ser definido.

BDD:

```
Cenário: Criar demanda para cliente
  Dado que existe um cliente cadastrado
  Quando crio uma demanda vinculada a esse cliente
  Então a demanda deve aparecer no detalhe do cliente
  E também na lista de demandas de clientes
```

US-0302 - Como usuário, quero acompanhar status e prioridade da demanda

Tasks:

- Criar status iniciais: aberta, em andamento, aguardando, concluída, cancelada.
- Criar prioridades: baixa, média, alta, urgente.
- Permitir alterar status e prioridade.
- Registrar mudanças no histórico.
- Exibir filtros por status, prioridade e responsável.

Critérios de aceitação:

- Usuário pode alterar status.
- Usuário pode alterar prioridade.
- Alterações ficam registradas.
- Filtros atualizam a listagem corretamente.

BDD:

```
Cenário: Alterar status da demanda
  Dado que uma demanda está aberta
  Quando altero o status para em andamento
  Então o novo status deve ser salvo
  E o histórico deve registrar a alteração
```

US-0303 - Como equipe, queremos comentar e anexar contexto na demanda

Tasks:

- Criar comentários internos.
- Criar checklist por demanda.
- Preparar modelo de anexos.
- Exibir atividade recente.

Critérios de aceitação:

- Comentários aparecem em ordem cronológica.
- Checklist pode ser marcado e desmarcado.

- Atividade recente mostra criacao, comentario e alteracoes.

BDD:

```
Cenario: Adicionar comentario interno
  Dado que estou no detalhe de uma demanda
  Quando adiciono um comentario interno
  Entao o comentario deve aparecer na linha do tempo
  E deve mostrar autor e data
```

Sprint 4 - Demandas Internas e Equipes

Objetivo: separar demandas internas das demandas de clientes e organizar o trabalho da equipe.

US-0401 - Como gestor, quero criar demandas internas sem cliente vinculado

Tasks:

- Reutilizar tabela de demandas com tipo internal_demand.
- Criar listagem separada de demandas internas.
- Criar campos: area, equipe, projeto, responsavel, prazo, prioridade.
- Criar detalhe da demanda interna.

Critérios de aceitacao:

- Demanda interna nao exige cliente.
- Demanda interna nao aparece na lista de demandas de clientes.
- Demanda interna pode ter equipe responsavel.

BDD:

```
Cenario: Criar demanda interna
  Dado que estou autenticado
  Quando crio uma demanda interna sem cliente
  Entao a demanda deve ser salva
  E deve aparecer somente na area de demandas internas
```

US-0402 - Como gestor, quero organizar demandas internas por equipe

Tasks:

- Criar cadastro de equipes.
- Vincular usuarios a equipes.
- Vincular demandas internas a equipes.
- Criar filtro por equipe.
- Criar visao de quadro por status.

Critérios de aceitacao:

- Gestor pode filtrar demandas por equipe.
- Demanda mostra responsavel e equipe.
- Quadro exibe demandas agrupadas por status.

BDD:

Cenário: Filtrar demandas por equipe
Dado que existem demandas em equipes diferentes
Quando filtro pela equipe Operacoes
Entao devo ver apenas demandas da equipe Operacoes

Sprint 5 - Distribuicao de Demandas

Objetivo: permitir que a empresa distribua trabalho com menos friccao.

US-0501 - Como gestor, quero atribuir demandas manualmente

Tasks:

- Criar seletor de responsavel.
- Validar se responsavel pertence a organizacao.
- Registrar historico de atribuicao.
- Notificar responsavel em UI.

Critérios de aceitacao:

- Gestor pode atribuir demanda a usuario.
- Usuario atribuido aparece na demanda.
- Historico registra responsavel anterior e novo.

BDD:

Cenário: Atribuir responsavel
Dado que uma demanda esta sem responsavel
Quando atribuo a demanda para Maria
Entao Maria deve aparecer como responsavel
E o historico deve registrar a atribuicao

US-0502 - Como gestor, quero criar regra simples de distribuicao

Tasks:

- Criar modelo de regra de distribuicao.
- Condições iniciais: tipo, prioridade, equipe, cliente VIP.
- Ações iniciais: atribuir equipe, atribuir responsavel, definir prioridade.
- Criar tela inicial de regras.
- Executar regra ao criar demanda.

Critérios de aceitacao:

- Regra ativa e aplicada ao criar demanda.
- Regra inativa nao e aplicada.
- Execucao da regra fica registrada.

BDD:

Cenário: Aplicar regra de distribuicao
Dado que existe uma regra ativa para demandas urgentes
Quando uma demanda urgente e criada
Entao a regra deve atribuir a demanda para a equipe configurada

Sprint Final - Integracoes, Personalizacao e Dashboard

Objetivo: concluir o escopo funcional planejado do CRM em uma unica sprint, juntando tudo que estava previsto nas sprints 6, 7 e 8. Esta sprint fecha o produto com integracoes para n8n/Make, personalizacao operacional e dashboard de indicadores.

Depois desta sprint, o foco deixa de ser adicionar grandes modulos e passa a ser estabilizacao: corrigir bugs, melhorar experiencia, refinar performance, endurecer seguranca, ajustar permissoes, documentar melhor e preparar o uso real da empresa.

Escopo consolidado:

- Integracoes outbound por webhooks.
- API inbound para n8n e Make.
- Chaves de API por organizacao.
- Campos personalizados para clientes.
- Status personalizados para demandas.
- Dashboard operacional com indicadores.
- Logs, auditoria e documentacao dos fluxos integrados.

US-0601 - Como administrador, quero cadastrar webhooks de saida

Tasks:

- Criar tabela de webhooks.
- Eventos iniciais: cliente.criado, cliente.atualizado, demanda.criada, demanda.atualizada, demanda.status_alterado.
- Criar tela de webhooks.
- Permitir ativar/desativar webhook.
- Criar assinatura simples por secret.
- Criar log de envio.

Critérios de aceitacao:

- Admin cadastra URL de webhook.
- Evento selecionado dispara envio para URL.
- Falhas ficam registradas.
- Webhook inativo nao recebe eventos.

BDD:

Cenario: Disparar webhook ao criar demanda
Dado que existe um webhook ativo para demanda.criada
Quando uma demanda e criada
Entao o CRM deve enviar o payload para a URL configurada
E deve registrar o resultado do envio

US-0602 - Como integrador, quero atualizar dados pelo n8n ou Make via API

Tasks:

- Criar chaves de API por organizacao.
- Criar endpoints para clientes.
- Criar endpoints para demandas.
- Validar escopo da chave.
- Registrar chamadas relevantes.
- Documentar exemplos para n8n e Make.

Critérios de aceitacao:

- API aceita requisicoes autenticadas por chave.
- Chave invalida recebe erro 401.
- Chave de uma organizacao nao acessa dados de outra.
- n8n/Make consegue criar ou atualizar demanda via HTTP request.

BDD:

Cenario: Criar demanda via API

Dado que possuo uma chave de API valida

Quando envio uma requisicao para criar demanda

Entao a demanda deve ser criada na organizacao correta

US-0701 - Como administrador, quero criar campos personalizados para clientes

Tasks:

- Criar definicao de campos personalizados.
- Tipos iniciais: texto, numero, data, booleano, selecao.
- Vincular campo a entidade cliente.
- Exibir campos no cadastro e detalhe do cliente.
- Validar campos obrigatorios.

Critérios de aceitacao:

- Admin cria campo personalizado.
- Campo aparece no formulario de cliente.
- Valor e salvo corretamente.
- Campo obrigatorio impede salvamento sem valor.

BDD:

Cenario: Campo personalizado obrigatorio

Dado que existe um campo personalizado obrigatorio em clientes

Quando tento salvar cliente sem preencher esse campo

Entao o CRM deve impedir o salvamento

E deve mostrar uma mensagem de validacao

US-0702 - Como administrador, quero personalizar status de demandas

Tasks:

- Criar modelo de workflow de demanda.
- Permitir status por tipo de demanda.
- Definir status inicial.

- Definir status finais.
- Migrar status fixos para configuracao.

Critérios de aceitacao:

- Admin configura status por tipo.
- Nova demanda usa status inicial configurado.
- Usuario so escolhe status validos para aquele tipo.

BDD:

Cenario: Usar status customizado

Dado que o admin configurou o status "Em revisao"

Quando altero uma demanda para esse status

Entao o CRM deve salvar o status customizado corretamente

US-0801 - Como gestor, quero ver indicadores principais do CRM

Tasks:

- Criar cards de total de clientes, demandas abertas, demandas vencidas e oportunidades.
- Criar grafico de demandas por status.
- Criar ranking de demandas por responsavel.
- Criar painel de atividades recentes.

Critérios de aceitacao:

- Dashboard carrega dados da organizacao atual.
- Indicadores refletem filtros basicos.
- Usuario ve somente dados permitidos.

BDD:

Cenario: Visualizar dashboard da organizacao

Dado que estou autenticado na organizacao A

Quando acesso o dashboard

Entao devo ver indicadores apenas da organizacao A

Critérios de pronto da Sprint Final

- Webhooks podem ser cadastrados, ativados, desativados e testados.
- Eventos relevantes disparam webhooks e geram log de entrega.
- API inbound permite criar/atualizar clientes e demandas com chave valida.
- Chaves de API sao escopadas por organizacao e podem ser revogadas.
- Campos personalizados aparecem nos formularios e respeitam obrigatoriedade.
- Status customizados podem ser configurados e usados em demandas.
- Dashboard mostra indicadores da organizacao ativa.
- Todas as novas rotas respeitam isolamento por organizacao.
- Todos os fluxos criticos possuem testes unitarios ou BDDs automatizados.
- CI/CD, build, lint, typecheck, scanner de segredos e audit passam antes do PR.

Fase seguinte - Estabilizacao e melhoria continua

Ao concluir a Sprint Final, o backlog principal passa a ser:

- Corrigir bugs encontrados em uso real.
 - Melhorar UX de telas com friccao.
 - Revisar mensagens de erro e estados vazios.
 - Reforcar permissoes e seguranca de API.
 - Melhorar performance de listagens e dashboards.
 - Ajustar responsividade mobile.
 - Criar documentacao operacional para equipe interna.
 - Preparar deploy, observabilidade e monitoramento.
-

Modelo de Dados Inicial

Entidades principais:

- organizations
- users
- organization_members
- teams
- team_members
- customers
- contacts
- notes
- demands
- demand_comments
- demand_checklist_items
- demand_history
- custom_field_definitions
- custom_field_values
- distribution_rules
- webhooks
- webhook_deliveries
- api_keys
- audit_events

Campos relevantes:

- Toda entidade operacional deve ter organization_id.
- Toda entidade relevante deve ter created_at e updated_at.
- Alteracoes importantes devem gerar audit_events ou historico especifico.
- Campos personalizados podem usar definicao relacional e valores flexiveis em jsonb quando fizer sentido.

Eventos Iniciais

- customer.created

- customer.updated
- contact.created
- demand.created
- demand.updated
- demand.status_changed
- demand.assigned
- demand.comment_created
- internal_demand.created
- webhook.delivery_failed

Integracao n8n e Make

Fluxo outbound:

CRM INTELIGENTE -> Evento interno -> Webhook -> n8n/Make -> Servico externo

Fluxo inbound:

Servico externo -> n8n/Make -> API CRM INTELIGENTE -> Cliente/Demanda/Comentario

Exemplos de automacao:

- Quando um formulario externo for preenchido, criar cliente e demanda.
- Quando uma demanda urgente for criada, enviar alerta no WhatsApp.
- Quando cliente ficar sem interacao por 15 dias, criar tarefa de contato.
- Quando demanda mudar para concluida, enviar pesquisa de satisfacao.
- Quando contrato for assinado, atualizar status do cliente.

Critérios Gerais de Pronto

- Funcionalidade implementada e testada localmente.
- Dados separados por organizacao.
- Erros tratados com mensagens claras.
- Rotas protegidas quando necessario.
- Migration criada quando houver alteracao de banco.
- Interface responsiva em desktop e mobile.
- Eventos relevantes registrados.
- Documentacao atualizada quando houver API, webhook ou configuracao.

Riscos e Mitigacoes

- Risco: personalizacao demais cedo demais.
Mitigacao: entregar campos personalizados primeiro para clientes e depois expandir.
- Risco: automacoes ficarem complexas dentro do CRM.
Mitigacao: usar n8n/Make para fluxos complexos e manter o CRM como origem de eventos e API confiavel.

- Risco: permissao multiusuario ficar fraca.
Mitigacao: modelar organizacao, membros e papeis desde a Sprint 1.
- Risco: desempenho em listagens grandes.
Mitigacao: paginacao, indices no Neon/PostgreSQL e filtros bem modelados desde o inicio.

Proximos Passos Imediatos

1. Garantir que `develop` esteja atualizado e que os bugfixes pendentes sejam integrados.
2. Abrir branch `feature/final-sprint`.
3. Implementar webhooks outbound e logs de entrega.
4. Implementar chaves de API e endpoints inbound para n8n/Make.
5. Implementar campos personalizados para clientes.
6. Implementar status personalizados para demandas.
7. Implementar dashboard operacional e indicadores.
8. Rodar testes, CI/CD, build e validacoes de seguranca.
9. Abrir PR da Sprint Final para `develop`.
10. Apos merge, iniciar ciclo de bugs, melhorias e estabilizacao.

